PLATAFORMA · VERSIÓN 1.0 · MAYO 2026

Documentación técnica de plataforma.

Arquitectura, stack tecnológico, modelo de datos, flujos de negocio, controles de seguridad y cumplimiento normativo de la tienda online y panel de gestión de Madcry Studio (madcry.com).

00 Datos del documento

Versión, alcance y responsables

DOCUMENTO	Documentación técnica de plataforma — madcry.com
VERSIÓN	1.0 · Mayo 2026
PROPIETARIO DEL PRODUCTO	Susana Madriz · Madcry Studio
IMPLEMENTACIÓN	Tane Solutions (consultora digital)
REPOSITORIO	`github.com/JohnSM7/tienda-chachimad` (privado)
ENTORNO PRODUCTIVO	https://madcry.com — Firebase Hosting (CDN global)
AUDIENCIA	Auditoría técnica, due diligence, evaluación legal.

↗ Tabla de contenidos

01 **Resumen ejecutivo** Qué es la plataforma y cómo está construida 03

02 **Stack tecnológico** Tecnologías y proveedores 04

03 **Arquitectura** Frontend, backend, infraestructura 05

04 **Modelo de datos** Esquema BD y datos personales 06

05 **Flujos críticos** Compra, contacto, publicación 07

06 **Seguridad — capas y controles** Defensa en profundidad 09

07 **Gestión de secretos** Credenciales y rotación 12

08 **SEO, rendimiento y accesibilidad** Indexación y CWV 14

09 **Cumplimiento legal** RGPD, LSSI-CE 15

10 **Operación, backups y continuidad** Despliegue y recuperación 16

11 **Anexos** Glosario, roles, registro 17

01 Resumen ejecutivo

Visión general en una página

Madcry Studio es la galería online y tienda directa de la artista **Susana Madriz**. La plataforma permite mostrar el catálogo de obra, gestionar la venta de piezas únicas (1/1) con cobro a tarjeta, recoger comisiones a medida y administrar todo el contenido desde un panel privado.

El sitio se construye con **Astro 5** en modo estático (SSG): cada despliegue genera HTML pre-renderizado servido desde una CDN global. La interactividad puntual se hidrata con **Vue 3**. El backend transaccional vive en **Supabase** (PostgreSQL gestionado, autenticación, almacenamiento y funciones serverless), mientras que el cobro lo realiza **Stripe**, sin que datos de tarjeta lleguen al servidor de Madcry.

CARACTERÍSTICAS CLAVE

SSG
PRE-RENDERING ESTÁTICO

PCI-DSS
PAGO DELEGADO STRIPE

RLS
ROW-LEVEL SECURITY ACTIVO

CSP
HEADERS DE SEGURIDAD

PRINCIPIOS RECTORES

Privilegio mínimo. Cada componente accede sólo a lo que necesita. La clave anónima del cliente no puede leer pedidos. Las credenciales potentes viven exclusivamente en funciones serverless.

Defensa en profundidad. Múltiples capas (CDN, CSP, RLS, JWT, validación servidor) impiden que el fallo de una sola comprometa el sistema.

Estático por defecto. Lo que se puede pre-renderizar, se pre-renderiza. Reduce superficie de ataque y coste a la vez.

Auditable. Todo el código está versionado en Git. Cada despliegue es trazable. Las funciones serverless registran logs estructurados.

- **Resultado de la auditoría interna (mayo 2026):** sin vulnerabilidades críticas. Mejoras menores de SEO y endurecimiento de CSP en cola. Detalle en sección 06.

02 Stack tecnológico

Componentes y proveedores en producción

FRONTEND Y COMPILACIÓN

FRAMEWORK
Astro 5
Generador estático con islas de hidratación. Build out: HTML + assets sin runtime servidor.

UI ISLAS
Vue 3
Composition API. Hidrata sólo el panel admin y el botón de compra.

ESTILOS
Tailwind 3
Utility-first. Sin CSS custom salvo animaciones globales.

BACKEND, DATOS Y SERVICIOS

BASE DE DATOS
Postgres 17
Supabase gestionado con RLS, vistas y triggers. Backups diarios.

AUTH
Supabase Auth
Magic-link por email para acceso al panel. Sin contraseñas.

STORAGE
Supabase Storage
Bucket `products`. Sólo admin escribe, lectura CDN.

FUNCIONES
Edge Functions
Deno. 7 funciones: pago, webhook, contacto, sync, rebuild, verify, shipping.

PAGO
Stripe Checkout
PCI-DSS Level 1 delegado. Webhooks firmados con HMAC-SHA256.

EMAIL
Resend
SPF/DKIM/DMARC alineados. Plantillas HTML del estudio.

INFRAESTRUCTURA Y DESPLIEGUE

CDN / HOSTING
Firebase Hosting
Multi-región, HTTPS forzado, HTTP/2, rollback instantáneo.

CI / CD
GitHub Actions
Workflow `deploy.yml` dispara build + deploy en cada push.

DNS
A definir
Apex apunta a Firebase. `www` redirige a apex.

¡ Todas las dependencias están **fijadas por versión**. La actualización se ejecuta de forma controlada cada trimestre.

03 Arquitectura

Visión por capas y separación de responsabilidades

La plataforma sigue una arquitectura **JAMstack con backend gestionado**: el cliente es estático, los servicios sensibles viven detrás de funciones serverless aisladas, y los proveedores externos se invocan desde el lado servidor con credenciales nunca expuestas al navegador.

DIAGRAMA DE CAPAS

CAPA 1 · CLIENTE
Navegador
- Astro HTML estático
- Vue 3 (islas)
- Sólo claves públicas
- CSP estricta + HSTS

CAPA 2 · BORDE
CDN + Funciones
- Firebase Hosting
- Edge Functions (Deno)
- JWT + CORS allowlist
- Rate-limit por IP

CAPA 3 · NÚCLEO
Datos y pago
- Postgres con RLS
- Stripe (PCI-DSS L1)
- Resend (email)
- Storage + bucket auth

▲ ▼ comunicación cifrada TLS 1.3

RELACIÓN ENTRE COMPONENTES

```
┌──────────┐    ┌──────────────┐    ┌──────────────┐
│ Visitante │──▶│ Firebase CDN │──▶│ HTML estático │
│ navegador │    │ (HTTPS+HSTS) │    │ de /dist/    │
└──────────┘    └──────────────┘    └──────────────┘
       │ click "comprar" / submit form
       ▼
┌──────────────────────────────────────────────────┐
│ Supabase Edge Functions (Deno · TLS · CORS allowlist) │
│ create-checkout · verify-session · stripe-webhook (HMAC) │
│ send-contact · admin-product-sync (JWT) · trigger-rebuild│
└──────────────────────────────────────────────────┘
       │              │              │
       ▼              ▼              ▼
┌──────────┐    ┌──────────┐    ┌──────────┐
│ Postgres │    │ Stripe   │    │ Resend   │
│ + RLS    │    │ Checkout │    │ SMTP API │
│ + Storage │    │ + webhooks │    │ SPF/DKIM │
└──────────┘    └──────────┘    └──────────┘
```

▣ **Lectura clave:** el navegador *nunca* habla directamente con Stripe ni con Postgres con privilegios elevados. Toda operación sensible pasa por una Edge Function que valida origen, autenticación y permisos.

04 Modelo de datos

Esquema PostgreSQL · públicas y privadas

Cuatro tablas centrales bajo el esquema `public`. Todas tienen **RLS activado** — Postgres aplica las políticas a cada consulta, independientemente del cliente.

TABLA	DATOS SENSIBLES	POLÍTICA RLS
`products` Catálogo	Slug, nombre, descripción, precio, imágenes, IDs Stripe.	**Lectura pública** sólo de productos no-borrador. **Escritura admin.**
`orders` Pedidos	Email cliente, nombre, teléfono, dirección, importe, IDs Stripe.	**Sin acceso público.** Sólo admin lee/escribe; webhooks usan service_role.
`messages` Contacto	Nombre, email, mensaje, IP, user-agent.	**Inserción pública. Lectura sólo admin.**
`categories` Colecciones	Slug, nombre, label, flag `hide_price`.	**Lectura pública.** Escritura admin.

VISTAS Y DERIVADOS

`products_public` filtra `status <> 'draft'` y une categoría. Es la única superficie expuesta al cliente para listados.

ALMACENAMIENTO DE ARCHIVOS

Bucket `products`, público en lectura, escritura sólo admin. MIME permitidos: JPEG, PNG, WebP, AVIF. Tamaño máx: **10 MB**. Cabecera `Cache-Control: 31536000`.

DATOS PERSONALES TRATADOS

CATEGORÍA	ORIGEN	BASE LEGAL	PLAZO
Datos de pedido	Stripe	Contrato	5 años (fiscal)
Mensajes de contacto	Form público	Interés legítimo	2 años
Email administrador	Supabase Auth	Servicio	Vigencia

05 Flujos críticos

Compra de obra, formulario de contacto, publicación admin

FLUJO 1 · COMPRA DE UNA PIEZA

```
[1]  Cliente entra en /product/<slug> (HTML estático CDN)
[2]  Click en "Adquirir Obra"
[3]  Navegador llama a create-checkout-session
       → CORS verificado contra allowlist
       → Lee precio FRESCO de la BD
       → Verifica status='available'
[4]  Crea Stripe Checkout Session con price_data inline
[5]  Pre-crea fila en orders con status='pending'
[6]  Devuelve URL · navegador redirige
[7]  Cliente paga en dominio Stripe (PCI-DSS)
[8]  Stripe llama al webhook stripe-webhook
       → Firma HMAC verificada
       → Marca pedido 'paid', producto 'sold'
       → Envía emails (Resend) cliente y admin
[9]  Cliente vuelve a /success?session_id=cs_...
[10] verify-session confirma con Stripe el pago
       → Devuelve email enmascarado (no completo)
```

✓ **Garantía técnica:** el precio nunca puede ser manipulado desde el navegador. La Edge Function relee el valor de Postgres en cada checkout.

FLUJO 2 · MENSAJE DE CONTACTO / COMISIÓN

```
[1]  Visitante envía formulario en /contact
[2]  Edge Function send-contact-email:
       → Honeypot anti-bots
       → Validación email + 5000 chars max
       → Rate-limit: 3/h email · 10/h IP
       → Inserta en tabla messages
       → Notifica a info@madcry.com
[3]  Confirmación al usuario (CORS allowlist)
```

FLUJO 3 · PUBLICACIÓN DE OBRA DESDE EL PANEL ADMIN

```
[1]  Susana abre /admin y solicita magic-link
[2]  Supabase Auth envía email con token de un solo uso
[3]  Click en el link → sesión JWT en el navegador
[4]  Carga del panel Vue (AdminPanel · island)
[5]  Edita / crea producto, sube imagen al bucket
[6]  POST con JWT a admin-product-sync
       → requireAdmin() verifica JWT y email
       → sanitizeImages() filtra schemas peligrosos
       → Crea/actualiza producto en BD
       → Sincroniza con Stripe Product
[7]  Click en "Publicar" → trigger-rebuild
       → Throttle de 60 s
       → Dispara workflow GitHub Actions
       → Build Astro + deploy Firebase (~2-3 min)
[8]  Sitio actualizado · siguiente visitante ve la obra
```

Inventario de funciones

Las 7 Edge Functions desplegadas

FUNCIÓN	AUTH	ORIGEN	PROPÓSITO
`create-checkout-session`	Pública	Cliente	Crea sesión Stripe con precio fresco.
`stripe-webhook`	HMAC	Stripe	Marca pedido pagado, manda emails.
`verify-session`	Pública	Cliente	Verifica estado de pago en Stripe.
`send-contact-email`	Pública	Cliente	Persiste mensaje y notifica admin.
`admin-product-sync`	JWT admin	Panel	CRUD productos + sync Stripe.
`send-shipping-email`	JWT admin	Panel	Notifica número de seguimiento.
`trigger-rebuild`	JWT admin	Panel	Lanza build Astro vía GH Actions.

06 Seguridad — capas y controles

Capa 1: red, transporte y cabeceras HTTP

Los controles se agrupan en **cuatro capas concéntricas**: red, transporte, aplicación y datos. Un fallo en una capa no compromete las siguientes.

CAPA DE RED Y TRANSPORTE

01	**HTTPS FORZADO**	Todo el tráfico circula sobre TLS 1.3. HTTP redirige a HTTPS por defecto.
02	**HSTS PRELOAD**	`max-age=31536000; includeSubDomains; preload` — el navegador rechaza HTTP futuro durante 1 año.
03	**CDN DISTRIBUIDA**	Mitigación implícita de DDoS volumétrico vía red global de Firebase.

CAPA DE APLICACIÓN · CABECERAS

04	**CONTENT-SECURITY-POLICY**	Política estricta: `default-src 'self'`; conexiones limitadas a Supabase/Stripe; `frame-ancestors 'none'` impide clickjacking.
05	**X-FRAME-OPTIONS DENY**	Refuerzo doble anti-clickjacking para navegadores antiguos.
06	**X-CONTENT-TYPE-OPTIONS**	`nosniff` — el navegador no interpreta MIME types incorrectos.
07	**REFERRER-POLICY**	`strict-origin-when-cross-origin` — no fuga de paths privados a terceros.
08	**PERMISSIONS-POLICY**	Bloquea cámara, micrófono, geolocalización; permite `payment` sólo a Stripe.
09	**CROSS-ORIGIN-OPENER-POLICY**	`same-origin` — aísla la pestaña principal de pop-ups maliciosos.

CAPA DE APLICACIÓN · API

10	**CORS ALLOWLIST**	Las funciones serverless sólo aceptan peticiones de `madcry.com` y entornos de desarrollo.
11	**VERIFICACIÓN JWT**	Funciones admin exigen JWT válido + email exacto del administrador.
12	**FIRMA STRIPE HMAC**	El webhook valida HMAC-SHA256 con `STRIPE_WEBHOOK_SECRET`.
13	**RATE-LIMIT POR IP Y EMAIL**	Contacto: 3/h email + 10/h IP. Rebuild: throttle 60 s.
14	**HONEYPOT ANTI-BOTS**	Campo invisible en el formulario. Reduce ~95% del spam automatizado.
15	**SANITIZACIÓN DE URLS**	Filtra esquemas `javascript:`, `data:`, `vbscript:` antes de Stripe / BD.

CAPA DE DATOS

16	**ROW-LEVEL SECURITY**	Postgres aplica políticas por fila. La clave anon **no puede** leer pedidos ni borradores.
17	**VISTA PÚBLICA LIMITADA**	`products_public` excluye productos en borrador.
18	**AISLAMIENTO SERVICE_ROLE**	Clave maestra sólo dentro de Edge Functions, nunca en navegador ni logs.
19	**MIME WHITELIST STORAGE**	Bucket `products` sólo acepta JPEG, PNG, WebP, AVIF. Tope 10 MB.
20	**CIFRADO EN REPOSO**	Postgres y Storage cifrados con AES-256 por la plataforma.
21	**EMAIL ENMASCARADO**	`verify-session` devuelve sólo iniciales+dominio.
22	**SOFT-DELETE PRODUCTOS**	No se borran filas: pasan a `draft` y Stripe se archiva.

PAGOS · SEPARACIÓN DE RESPONSABILIDADES

▣ Madcry Studio **nunca** almacena ni recibe datos de tarjeta. El proceso de cobro ocurre íntegramente en infraestructura de Stripe (PCI-DSS Level 1). Madcry sólo recibe del webhook el resultado del pago, los datos de envío del cliente y el identificador opaco `cs_…` de la sesión.

RESUMEN — AUDITORÍA INTERNA 2026-05

ÁREA	ESTADO	NOTAS
Secretos en repositorio	OK	Verificado: ningún `.env` en historial Git.
RLS en tablas sensibles	OK	4/4 tablas con RLS activo y políticas explícitas.
Webhook Stripe firmado	OK	HMAC-SHA256 con secret rotable.
CORS de Edge Functions	OK	Allowlist tras endurecimiento de mayo 2026.
Rate-limit en formularios	OK	Honeypot + límites por email/IP.
Sanitización de inputs	OK	URLs filtradas, validación tipos.
CSP sin `unsafe-inline`	MEJORA	Pendiente migración a hashes/nonces.
Captcha en formularios	MEJORA	Cloudflare Turnstile en cola.

✓ **Conclusión:** sin hallazgos críticos. Las dos mejoras señaladas son endurecimiento adicional, no riesgos activos. Honeypot y rate-limit cubren el riesgo actual de spam; la CSP estricta vigente ya bloquea las clases más comunes de XSS.

07 Gestión de secretos

Inventario y procedimiento de rotación

Las credenciales se clasifican en **dos grupos** según su exposición: las que viajan en el bundle del navegador y las que viven exclusivamente en infraestructura servidor.

PÚBLICAS (VISIBLES EN BUNDLE DEL NAVEGADOR)

VARIABLE	ORIGEN	RIESGO
`PUBLIC_SUPABASE_URL`	Supabase	Nulo · es URL pública del proyecto.
`PUBLIC_SUPABASE_ANON_KEY`	Supabase	Bajo · sólo permite lo que las políticas RLS permitan.
`PUBLIC_STRIPE_PUBLISHABLE_KEY`	Stripe	Nulo · diseñada para ir en cliente.

PRIVADAS (SÓLO EN EDGE FUNCTIONS / GITHUB SECRETS)

VARIABLE	ALMACENAMIENTO	PROCEDIMIENTO DE ROTACIÓN
`SUPABASE_SERVICE_ROLE_KEY`	Edge Functions	Dashboard → Settings → API → Reset.
`STRIPE_SECRET_KEY`	Edge Functions	Stripe → Developers → API Keys → Roll.
`STRIPE_WEBHOOK_SECRET`	Edge Functions	Stripe → Webhooks → Endpoint → Roll.
`RESEND_API_KEY`	Edge Functions	Resend → API Keys → Revoke + recreate.
`SUPABASE_DB_PASSWORD`	Sólo terminal puntual	Supabase → Database → Reset password.
`GITHUB_TOKEN`	Edge Functions	GitHub Settings → Developer → Tokens.
`FIREBASE_SERVICE_ACCOUNT`	GitHub Actions secrets	Firebase IAM → service account → New key.

REGLAS INTERNAS DE MANEJO

- [ ] Nunca pegar una credencial privada en chat, email, ticket o screenshot.
- [ ] Nunca persistirla en `.env.local` salvo si es estrictamente `PUBLIC_*`.
- [ ] Si es necesaria de forma puntual, exportarla en la sesión de terminal y borrar el histórico (`history -c`).
- [ ] Rotar inmediatamente cualquier credencial que haya estado en un dispositivo perdido, repositorio público o canal compartido.
- [ ] Toda rotación queda registrada con fecha en este documento (Anexos · Registro).

> ! **Política de mínimo necesario:** el archivo `.env.local` que vive en la máquina de desarrollo contiene exclusivamente claves `PUBLIC_*`. Las claves potentes están en Supabase secrets — accesibles a la persona con login del proyecto, no al sistema de archivos del developer.

PROCEDIMIENTO DE ROTACIÓN PERIÓDICA

FRECUENCIA	ÁMBITO	RESPONSABLE
Trimestral	Revisión de tokens largos (GitHub, service accounts).	Tane Solutions
Anual	Rotación completa de todas las claves privadas.	Tane Solutions
Inmediata	Tras cualquier sospecha de exposición o salida de personal.	Tane Solutions + Madcry

08 SEO, rendimiento y accesibilidad

Indexación, Core Web Vitals y WCAG

SEO

CONTROL	ESTADO
Sitemap automático (`sitemap-index.xml`)	ACTIVO · 30 URLs.
`robots.txt` con directivas explícitas	OK · admin/success bloqueados.
Datos estructurados JSON-LD	OK · Organization, WebSite, Product, Article, Breadcrumb, Event.
Meta description única por página	OK · 11 páginas.
`rel="canonical"` global	OK
Open Graph + Twitter Card 1200×630	OK
`noindex` en /admin, /success, /404	OK

RENDIMIENTO

- **SSG (pre-render):** el HTML llega listo desde la CDN.
- **Caché agresiva:** assets versionados con `max-age=31536000, immutable`.
- **Lazy-loading** en imágenes de listado.
- **Preload + onload-swap** para fuentes (no bloquea render).
- **HTTP/2** y **Brotli** servidos por Firebase.

ACCESIBILIDAD

El sitio sigue las pautas **WCAG 2.1 AA** en navegabilidad: contrastes verificados, estructura semántica, focus visibles, atributos `alt` en imágenes, formulario etiquetado. Auditoría de profundidad con axe-core prevista.

09 Cumplimiento legal

RGPD, LSSI-CE y normativa aplicable

RGPD (UE 2016/679)

REQUISITO	IMPLEMENTACIÓN
Información clara al usuario	Páginas `/privacidad` y `/legal` en footer.
Base legal documentada	Contrato (pedidos), interés legítimo (contacto), consentimiento (futuro).
Minimización de datos	Sólo lo imprescindible para envío y facturación.
Derechos ARSULIPO	Por email a `info@madcry.com`.
Cifrado	TLS 1.3 + AES-256 por proveedor.
Encargados de tratamiento	Supabase, Stripe, Resend, Firebase. DPA + SCC firmados.
Plazos	5 años (datos fiscales), 2 años (mensajes).
Brechas de seguridad	Notificación AEPD < 72 h.

LSSI-CE

- Aviso legal con titularidad, NIF, dirección y email de contacto.
- Política de cookies (en cola — sin cookies de terceros sin consentimiento).
- Identificación clara de precio, IVA, gastos de envío y forma de pago.
- Confirmación de pedido por email con resumen y referencia.

COMERCIO ELECTRÓNICO

Cumple con los **14 días de derecho de desistimiento** (TRLGDCU), salvo en piezas personalizadas (comisión a medida) donde el desistimiento queda excluido por su naturaleza específica conforme al art. 103.c). Esta excepción se informa antes de finalizar el encargo.

10 Operación, backups y continuidad

Despliegue, respaldo y recuperación

DESPLIEGUE

Cada commit en `main` dispara `deploy.yml` de GitHub Actions: `npm ci`, `astro build` y deploy a Firebase Hosting. Tiempo medio: 2-3 minutos. También se dispara desde el panel admin (función `trigger-rebuild`).

BACKUPS

COMPONENTE	FRECUENCIA	RETENCIÓN	MECANISMO
PostgreSQL	Diaria automática	7 d (free) / 30 d (Pro)	Supabase Backups
Imágenes (Storage)	Continuo	Indefinido	Replicación interna
Código	En cada commit	Indefinido	GitHub distribuido
Versión desplegada	En cada build	10 versiones	Firebase rollback

RECUPERACIÓN ANTE INCIDENCIAS

ESCENARIO	PROCEDIMIENTO	RTO OBJETIVO
Bug en producción	Rollback en Firebase a versión anterior	< 5 min
Pérdida de datos en BD	Restauración de backup + replay de webhooks	< 4 h
Compromiso de credencial	Rotación + auditoría de logs	< 1 h
Caída del proveedor	Comunicación al cliente · esperar SLA	Según SLA

MONITORIZACIÓN

Logs estructurados en cada Edge Function (Supabase Dashboard). Stripe envía email automático ante webhooks fallidos. Resend reporta entrega/rebote. Próxima iteración: integración con **Sentry** y agregación en una sola consola.

11 Anexos

Glosario y matriz de roles

A · GLOSARIO RÁPIDO

SSG	Static Site Generation. El sitio se compila a HTML estático en build, no en cada petición.
RLS	Row-Level Security. Postgres aplica políticas por fila a cualquier consulta.
JWT	JSON Web Token. Identidad firmada que el navegador presenta para autenticarse.
HMAC	Firma de un mensaje con un secreto compartido para verificar su origen.
CSP	Content-Security-Policy. Limita qué recursos puede cargar el navegador.
HSTS	HTTP Strict Transport Security. Fuerza HTTPS futuras visitas.
PCI-DSS	Estándar de seguridad para datos de tarjeta. Lo cumple Stripe en nombre de Madcry.
JAMSTACK	Arquitectura cliente estático + servicios vía API.

B · MATRIZ DE ROLES

ROL	QUIÉN	ACCESO
Visitante anónimo	Cualquiera	Catálogo público, blog, compras, contacto.
Cliente con pedido	Comprador	/success de su pedido, emails de confirmación y envío.
Administradora	Susana Madriz (info@madcry.com)	Panel /admin: CRUD productos, mensajes, publicar, shipping.
Mantenimiento técnico	Tane Solutions	Repo Git, Supabase, Stripe, Firebase (acceso revocable).

C · REGISTRO DE AUDITORÍAS Y ROTACIONES

FECHA	EVENTO	RESPONSABLE
2026-05-04	Auditoría interna de seguridad y SEO. Endurecimiento de CORS, sanitización de imágenes, throttle en rebuild, sitemap automático, JSON-LD, descripciones únicas. Sin hallazgos críticos.	Tane Solutions
2026-05-04	Limpieza de `.env.local` · retiradas service_role, stripe_secret, webhook_secret, resend_key, db_password.	Tane Solutions
2026-05-04	Generación de imagen OG 1200×630 vía puppeteer + plantilla HTML del estudio.	Tane Solutions
— próxima	*Rotación trimestral programada de tokens y revisión de dependencias.*	*Tane Solutions*

▣ **Documento controlado.** Este documento se versiona junto al código en `docs/`. Cualquier cambio en la arquitectura, dependencia o control de seguridad debe reflejarse aquí en la misma pull request.

CONTACTO TÉCNICO

PROPIETARIO DEL PRODUCTO	Susana Madriz · `info@madcry.com`
MANTENIMIENTO	Tane Solutions · `www.tanesolutions.com`
REPOSITORIO	`github.com/JohnSM7/tienda-chachimad`

Plataforma construida con criterio.

Cualquier consulta sobre arquitectura, seguridad o cumplimiento se responde por escrito por la entidad responsable del mantenimiento. Datos de contacto al pie.